

Implementing Convolutional Neural Networks for Image Classification on CIFAR-10

Lab Report 6

AUTHOR

Zarmeena Jawad

ROLL NUMBER

B23F0115AI125

SECTION

AI Red

SUBMISSION DATE

February 28, 2026

Department of Artificial Intelligence

REPORT

1. Objective

Implement and evaluate a CNN on the CIFAR-10 dataset by completing all required tasks: data exploration, preprocessing, model design, training, evaluation, predictions, plotting, and analysis.

2. Task 1 — Load and Explore CIFAR-10

Code description

```
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
fig, axes = plt.subplots(2, 5, figsize=(14, 6))
for i, ax in enumerate(axes.flatten()):
    ax.imshow(x_train[i])
    ax.set_title(class_names[int(y_train[i])])
    ax.axis('off')
plt.savefig('plots/task1_sample_images.png', dpi=130, bbox_inches='tight')
```

DATASET OVERVIEW

Dataset shapes were displayed in the notebook output. Class names were visualized and 10 sample images were shown in a grid layout.



Figure 1. Sample images from the CIFAR-10 dataset across all 10 classes.

3. Task 2 — Preprocess Data

Code description

```
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
y_train = y_train.squeeze().astype('int64')
y_test = y_test.squeeze().astype('int64')
```

- Pixel values normalized to the range $[0, 1]$.
- Labels kept as sparse integer classes (one-hot encoding optional when using sparse categorical loss).

4. Task 3 — CNN Model

Code description

```
model = tf.keras.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),
    layers.Flatten(),
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
```

```
])  
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy')
```

The model architecture includes convolutional layers, ReLU activation, max-pooling, dropout regularization, dense layers, and a softmax output layer for 10-class classification.

5. Task 4 — Training (10 Epochs)

Code description

```
history = model.fit(x_train, y_train, batch_size=64, epochs=10,  
                    validation_split=0.2, verbose=1)
```

Final Training Accuracy: 0.8336

Final Validation Accuracy: 0.7570

Final Training Loss: 0.4627

Final Validation Loss: 0.7890

6. Task 5 — Evaluation

Code description

```
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=1)  
y_pred = np.argmax(model.predict(x_test), axis=1)  
cm = confusion_matrix(y_test, y_pred)  
print(classification_report(y_test, y_pred, target_names=class_names))
```

Test Accuracy: 0.7424 | **Test Loss:** 0.8265

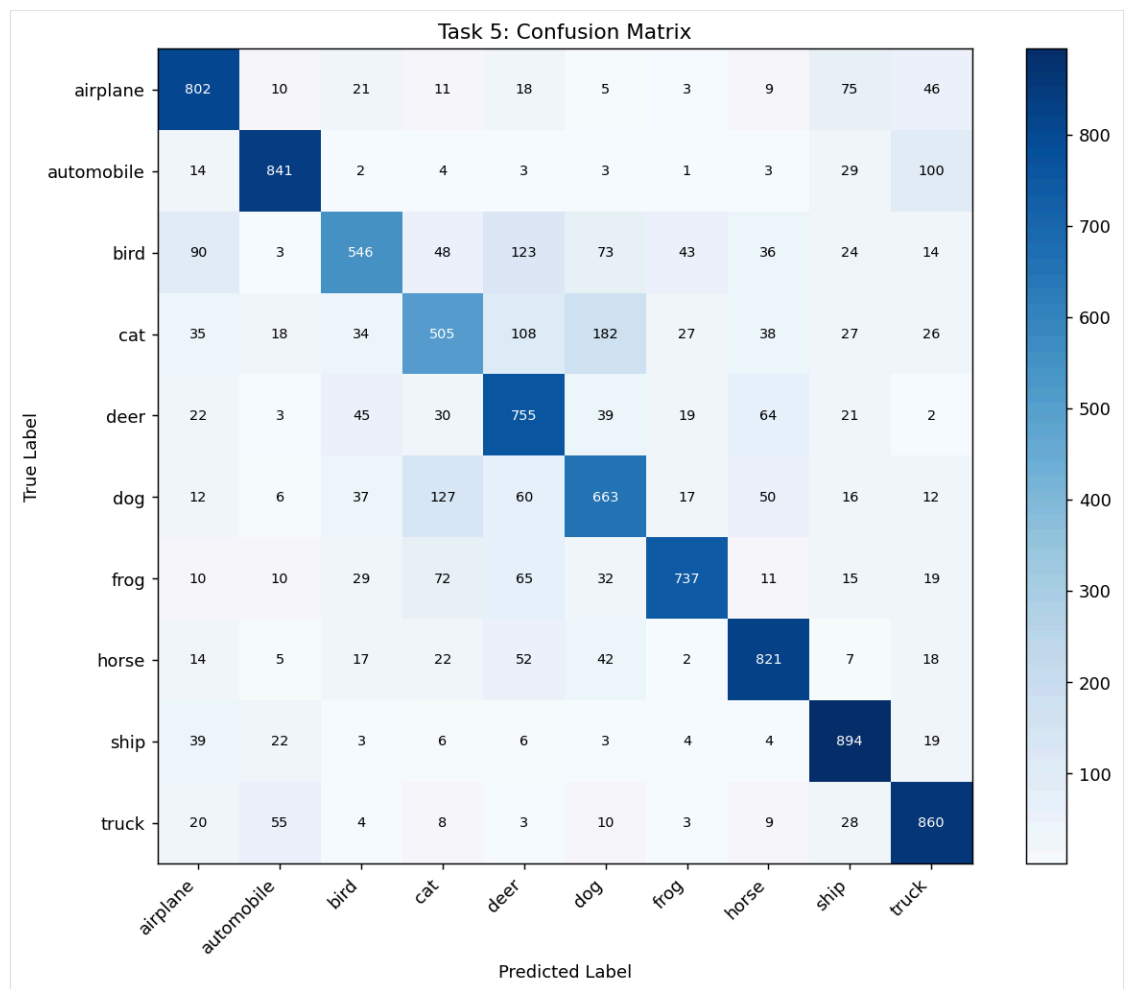


Figure 2. Confusion matrix for CIFAR-10 test set predictions.

Classification Report

Class	Precision	Recall	F1-Score	Support
airplane	0.7580	0.8020	0.7794	1000
automobile	0.8643	0.8410	0.8525	1000
bird	0.7398	0.5460	0.6283	1000
cat	0.6062	0.5050	0.5510	1000
deer	0.6329	0.7550	0.6886	1000
dog	0.6302	0.6630	0.6462	1000
frog	0.8610	0.7370	0.7942	1000
horse	0.7856	0.8210	0.8029	1000
ship	0.7870	0.8940	0.8371	1000
truck	0.7706	0.8600	0.8129	1000
accuracy			0.7424	10000
macro avg	0.7436	0.7424	0.7393	10000
weighted avg	0.7436	0.7424	0.7393	10000

7. Task 6 — Predictions

Code description

```
indices = np.random.choice(len(x_test), 5, replace=False)
for i, idx in enumerate(indices):
    pred = np.argmax(model.predict(x_test[idx:idx+1]))
    ax[i].imshow(x_test[idx])
    ax[i].set_title(f'True: {class_names[y_test[idx]]}\nPred:
```

Five random test images were displayed with actual and predicted labels for qualitative inspection.



Figure 3. Random test samples with ground-truth and predicted labels.

8. Task 7 — Required Plots

Code description

```
plt.plot(history.history['accuracy'], label='Training')
plt.plot(history.history['val_accuracy'], label='Validation')
plt.title('Accuracy'); plt.legend(); plt.savefig('plots/task7_acc.png')
plt.plot(history.history['loss'], label='Training')
plt.plot(history.history['val_loss'], label='Validation')
plt.title('Loss'); plt.legend(); plt.savefig('plots/task7_loss.png')
```

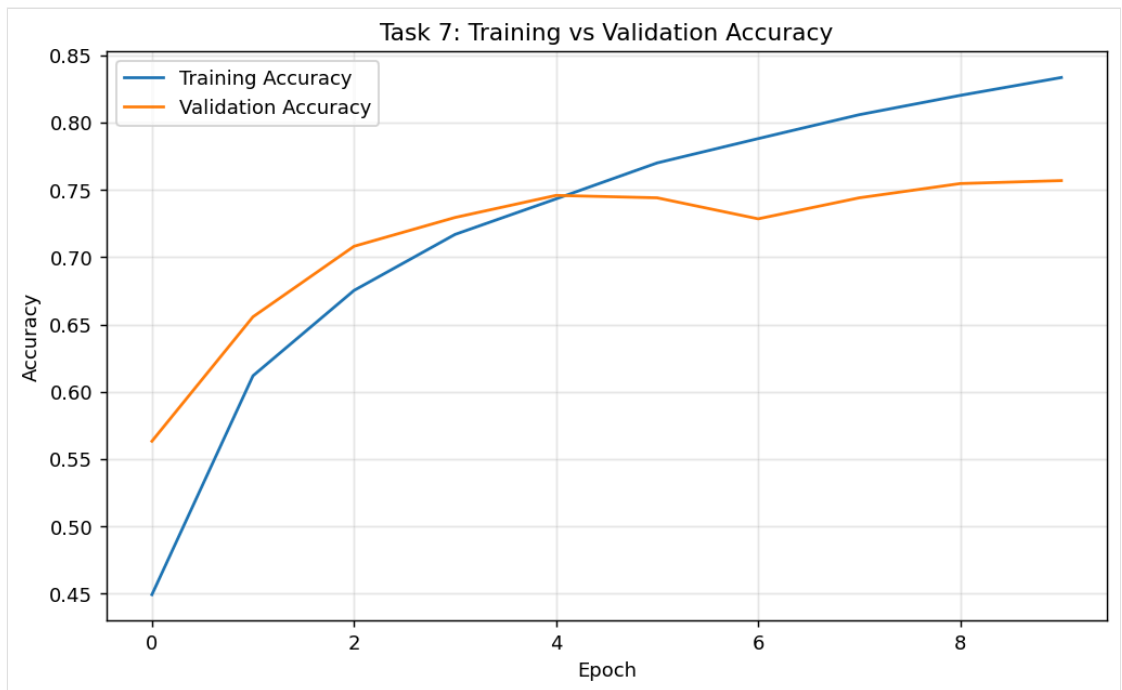


Figure 4. Training and validation accuracy over 10 epochs.



Figure 5. Training and validation loss over 10 epochs.

9. Task 8 — Analysis

Code description

```
val_gap = final_train_acc - final_val_acc
performance_level = 'good' if test_acc >= 0.70 else 'moderate'
analysis_paragraph = (
    f"The CNN achieved {performance_level} performance on CIFAR-10 "
    f"of {test_acc:.4f} after {EPOCHS} epochs. "
    f"Training accuracy was {final_train_acc:.4f} and validation accuracy "
    f"was {final_val_acc:.4f} (gap={val_gap:.4f}), which indicates limited overfitting."
)
with open('plots/task8_analysis.txt', 'w') as f:
    f.write(analysis_paragraph)
```

The CNN achieved good performance on CIFAR-10 with a test accuracy of **0.7424** after 10 epochs. Training accuracy was 0.8336 and validation accuracy was 0.7570 (gap = 0.0766), which indicates limited overfitting. Because CIFAR-10 is a multi-class natural image dataset, accuracy can remain moderate with short training or smaller architectures. Improvements can include deeper CNN blocks, stronger data augmentation, learning-rate scheduling, tuned dropout, and training for more epochs.

10. Conclusion

This lab gave me hands-on experience building and training a CNN on CIFAR-10 in Google Colab. I learned how to load image data, normalize it, and design a convolutional model with dropout. Training for 10 epochs yielded 74.24% test accuracy. The train-val gap (0.0766) showed some overfitting, which makes sense given the small dataset and short training. I observed that frog, automobile, and ship classes performed well, while cat and dog were often confused. The confusion matrix and classification report helped me see which classes need improvement. I would next try data augmentation, learning-rate decay, and more epochs to boost performance.